



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 3	Tema: Introducción a React
	Semana 10	Tutor: Ing. Victoria Castro

Introducción:

En el desarrollo de software tradicional, gestionar el estado de la interfaz de usuario (UI) solía ser una tarea imperativa y propensa a errores. Tenías que decirle al navegador exactamente *cómo* cambiar cada elemento (por ejemplo: "busca este botón, cambia su color, luego actualiza este texto").

React revoluciona esto al implementar tres principios fundamentales de ingeniería:

1. Declaratividad: Tú describes *qué* quieres ver en pantalla según el estado actual, y React se encarga de actualizar el DOM de forma eficiente.

2. Componentización: Fomenta la modularidad y la reutilización de código, dividiendo la UI en piezas independientes y aisladas.
3. Predictibilidad: Gracias al flujo de datos unidireccional, es mucho más fácil rastrear errores y predecir cómo se comportará la aplicación.

Sección 1: Creación y anidamiento de componentes

Desde el punto de vista arquitectónico, los componentes permiten aplicar el principio de Responsabilidad Única (SRP). En lugar de tener un archivo HTML de miles de líneas, dividimos la interfaz en piezas lógicas.

Composición sobre Herencia

React utiliza un modelo de composición. Esto significa que los componentes pueden contener a otros componentes, permitiendo construir jerarquías complejas a partir de piezas simples.

Ejemplo: **Estructura de un Perfil de Usuario**

Imagina que estamos construyendo una red social. No creamos un solo componente "TodoElPerfil", sino que anidamos varios:

// 1. Componente de bajo nivel (Atómico)

```
function Avatar() {  
  return (  
      
  );  
}
```

// 2. Componente de nivel medio (Molécula)

```
function PerfilCard() {  
  return (  
    <section className="card">  
      <Avatar />  
    </section>  
  );  
}
```

```

    <div className="info">
      <h3>Lin Lanying</h3>
      <p>Ingeniera de Software</p>
    </div>
  </section>
);
}

// 3. Componente de alto nivel (Organismo / Página)
export default function Galeria() {
  return (
    <nav>
      <h1>Directorio de Expertos</h1>
      <div className="flex-container">
        <PerfilCard />
        <PerfilCard />
        <PerfilCard />
      </div>
    </nav>
  );
}

```

Por qué las mayúsculas son obligatorias: En la ingeniería detrás de React, cuando el compilador (Babel) ve `<div />`, sabe que es una etiqueta HTML estándar. Pero cuando ve `<PerfilCard />`, entiende que es una referencia a una función que tú definiste, y debe ejecutar esa función para saber qué renderizar.

Sección 2: Escribir marcado con JSX

JSX no es un string ni es HTML; es una **extensión sintáctica de JavaScript**. Cada etiqueta de JSX se convierte realmente en una llamada a una función llamada `React.createElement()`.

El concepto del "Fragmento" (`<>...</>`)

En ingeniería de sistemas, una función solo puede devolver un valor a la vez. Como un componente de React es una función, no puede devolver "dos cosas" por separado.

El problema:

// ERROR: La función intenta devolver dos objetos distintos

```
function ErrorComponent() {  
  return (  
    <h1>Título</h1>  
    <p>Párrafo</p>  
  );  
}
```

La solución (Fragment): El uso de `<> ... </>` permite agrupar elementos sin añadir nodos innecesarios al DOM real del navegador, manteniendo el árbol de nodos limpio y eficiente.

Lógica de JavaScript dentro de JSX (Las llaves `{}`)

La verdadera potencia de JSX es que puedes "escapar" a JavaScript puro en cualquier momento usando llaves. Esto permite que el marcado sea dinámico.

Ejemplo: Atributos dinámicos y estilos

```
function PerfilPersonalizado() {  
  const usuario = {  
    nombre: 'Gregorio Y. Zara',  
    fotoUrl: 'https://i.imgur.com/7vQDOFPs.jpg',  
    tamanoFoto: 90,  
  };  
  
  return (  
    <div className="container">  
      <h1>{usuario.nombre}</h1>
```

```

<img
  className="avatar"
  src={usuario.fotoUrl}
  alt={`Foto de ` + usuario.nombre}
  style={{
    width: usuario.tamanoFoto,
    height: usuario.tamanoFoto,
    borderRadius: '50%'
  }}
/>
</div>
);
}

```

Desglose técnico de las Reglas de Oro:

1. **CamelCase en Atributos:** En JavaScript, `class` es una palabra reservada para definir clases de objetos. Por eso, para definir clases de CSS, usamos `className`. Lo mismo ocurre con `onclick` (HTML) vs `onClick` (React/JSX).
2. **Cierre estricto:** En HTML, podías dejar un `<input>` abierto. En JSX, esto rompería la compilación. El cierre automático (`<img />`) es obligatorio para mantener la integridad del árbol de sintaxis abstracta (AST).
3. **El objeto style:** Nota en el ejemplo anterior que usamos `style={{...}}`. Las primeras llaves son para "entrar a JavaScript" y las segundas son para definir un **objeto** de JavaScript. A diferencia de HTML (donde el estilo es un string), en React es un objeto, lo que facilita cambiar estilos mediante código de forma programática.

Sección 3: Renderizado condicional y de listas

React te permite usar toda la potencia de JavaScript para decidir qué mostrar.

Condicionales

Puedes usar la sintaxis habitual de `if` o el operador ternario:

```

<div>
  {isLoggedIn ? <AdminPanel /> : <LoginForm />}
</div>

```

Listas

Para renderizar múltiples componentes similares, usamos la función `.map()` de los arreglos. **Importante:** Cada elemento de la lista debe tener una propiedad `key` única para que React pueda optimizar las actualizaciones de renderizado.

```
const productos = [  
  { title: 'Col', id: 1 },  
  { title: 'Ajo', id: 2 },  
];  
  
const listItems = productos.map(producto =>  
  <li key={producto.id}>  
    {producto.title}  
  </li>  
);
```

Sección 4: Responder a eventos y actualizar la pantalla

Para que una aplicación sea interactiva, necesita manejar eventos y "recordar" cosas. Aquí es donde entra el concepto de **Estado (State)**.

Manejo de eventos

Declaras funciones de manejo de eventos dentro de tus componentes:

```
function MiBoton() {  
  function manejarClick() {  
    alert('¡Me hiciste click!');  
  }  
  
  return <button onClick={manejarClick}>Haz click aquí</button>;  
}
```

El Hook `useState`

Si quieres que un componente cambie su información en pantalla, necesitas usar el Hook `useState`.

```
import { useState } from 'react';

function Contador() {
  const [cuenta, setCuenta] = useState(0);

  function incrementar() {
    setCuenta(cuenta + 1);
  }

  return (
    <button onClick={incrementar}>
      Has hecho click {cuenta} veces
    </button>
  );
}
```

Sección 5: Compartir datos entre componentes

En la ingeniería de software, la gestión del flujo de datos es vital. En React, los datos fluyen de padres a hijos a través de **props**.

Si necesitas que dos componentes compartan el mismo estado (por ejemplo, dos botones que muestran el mismo número), debes "**eleva el estado**" al componente padre común más cercano.

Bibliografía:

Para profundizar consulta la documentación en: <https://es.react.dev/learn>